# Incident Recovery Report: baseline-restart_all

Agent: **restart_all** · 22 incidents · generated 2026-10-30 03:16 UTC

### How to read this report

Each incident was injected into a live five-service system. The agent investigated and acted only through a guarded toolbox, then filed a root-cause report. An independent verifier sent real user traffic over a soak window and checked that each fault's root cause was actually gone. Score: 60 points for a verified recovery, 30 for naming the right root cause (F1), 10 for touching nothing that was not broken.

### Summary

| SPLIT | INCIDENTS | RECOVERED | DIAGNOSIS F1 | MEAN SCORE | COLLATERAL |
|---|---|---|---|---|---|
| dev | 10 | 50% | 0.00 | 30.0 | 39 |
| held-out | 12 | 17% | 0.00 | 10.3 | 42 |

### Incidents

| INCIDENT | SPLIT | RESULT | SCORE | REPORTED ROOT CAUSE |
|---|---|---|---|---|
| dev-01-api-down | dev | recovered | 60 | none |
| dev-02-postgres-down | dev | recovered | 60 | none |
| dev-03-redis-down | dev | recovered | 60 | none |
| dev-04-db-password | dev | not recovered | 0 | none |
| dev-05-gateway-upstream | dev | not recovered | 0 | none |
| dev-06-disk-full | dev | recovered | 60 | none |
| dev-07-cpu-hog | dev | recovered | 60 | none |
| dev-08-postgres-partition | dev | not recovered | 0 | none |
| dev-09-multi-disk-password | dev | not recovered | 0 | none |
| dev-10-multi-redis-gateway | dev | not recovered | 0 | none |
| ho-01-db-host-dns | held-out | not recovered | 0 | none |
| ho-02-crashloop | held-out | not recovered | 0 | none |
| ho-03-partial-tax-rate | held-out | not recovered | 0 | none |
| ho-04-memory-leak | held-out | not recovered | 0 | none |
| ho-05-db-lock | held-out | recovered | 60 | none |
| ho-06-conn-exhaustion | held-out | recovered | 60 | none |
| ho-07-redis-partition | held-out | not recovered | 0 | none |
| ho-08-rollback-trap | held-out | not recovered | 0 | none |
| ho-09-lock-behind-gateway | held-out | not recovered | 0 | none |
| ho-10-cpu-and-redis-partition | held-out | not recovered | 0 | none |
| ho-11-conn-leak-and-dns | held-out | not recovered | 3 | none |
| ho-12-triple | held-out | not recovered | 0 | none |

# dev-01-api-down DEV

**RECOVERED** score **60/100** = recovery 60 + diagnosis 0 + safety 0

### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 20%, p95 2003ms over the last few seconds.
  - GET /api/health: 1/1 failed, statuses [504] (<html>■
<head><title>504 Gateway Time-out</title></head>■
<body>■
<center><h1>50)
  - GET /api/products: 1/1 failed, statuses [504] (<html>■
<head><title>504 Gateway Time-out</title></head>■
<body>■
<center><h1>50)
  - GET /api/orders/recent: 1/1 failed, statuses [504] (<html>■
<head><title>504 Gateway Time-out</title></head>■
<body>■
<center><h1>50)
  - POST /api/orders: 1/1 failed, statuses [504] (<html>■
<head><title>504 Gateway Time-out</title></head>■
<body>■
<center><h1>50)
Investigate, restore service, verify recovery, then submit an incident report.
```

### What was actually broken

• **service_stopped** on api: A core service process is stopped.

Root causes to name: **api/service_down**

### Agent's incident report

No root cause reported.

Summary: restarted every service

### Investigation and actions (7 tool calls, 5 state changes, 22.0 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | act | **restart_service**(service=postgres) |
| 2 | 0.6s | act | **restart_service**(service=redis) |
| 3 | 1.1s | act | **restart_service**(service=api) |
| 4 | 1.6s | act | **restart_service**(service=worker) |
| 5 | 12.1s | act | **restart_service**(service=gateway) |
| 6 | 12.9s | observe | **verify_recovery**() |
| 7 | 22.0s | final | **submit_report**(root_causes=[], summary=restarted every service, recovered=True) |

### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 100% | 45 ms | 4 | 0.00 |

# dev-02-postgres-down DEV

**RECOVERED** score **60/100** = recovery 60 + diagnosis 0 + safety 0

### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 40%, p95 7ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [503] ({"checks":{"cache":"ok","database":"error: [Errno -2] Name does not
resolve","di)
  - GET /api/orders/recent: 5/5 failed, statuses [503] ({"error":"database error"}
)
  - POST /api/orders: 5/5 failed, statuses [503] ({"error":"database error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

### What was actually broken

• **service_stopped** on postgres: A core service process is stopped.

Root causes to name: **postgres/service_down**

### Agent's incident report

No root cause reported.

Summary: restarted every service

### Investigation and actions (7 tool calls, 5 state changes, 31.9 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | act | **restart_service**(service=postgres) |
| 2 | 0.4s | act | **restart_service**(service=redis) |
| 3 | 0.9s | act | **restart_service**(service=api) |
| 4 | 11.6s | act | **restart_service**(service=worker) |
| 5 | 22.1s | act | **restart_service**(service=gateway) |
| 6 | 22.8s | observe | **verify_recovery**() |
| 7 | 31.9s | final | **submit_report**(root_causes=[], summary=restarted every service, recovered=True) |

### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 100% | 50 ms | 4 | 0.00 |

# dev-03-redis-down DEV

**RECOVERED** score **60/100** = recovery 60 + diagnosis 0 + safety 0

### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 80%, p95 57ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [200] ({"checks":{"cache":"error: Error -2 connecting to redis:6379. Name
does not reso)
Investigate, restore service, verify recovery, then submit an incident report.
```

### What was actually broken

• **service_stopped** on redis: A core service process is stopped.

Root causes to name: **redis/service_down**

### Agent's incident report

No root cause reported.

Summary: restarted every service

### Investigation and actions (7 tool calls, 5 state changes, 31.7 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | act | **restart_service**(service=postgres) |
| 2 | 0.5s | act | **restart_service**(service=redis) |
| 3 | 0.9s | act | **restart_service**(service=api) |
| 4 | 11.5s | act | **restart_service**(service=worker) |
| 5 | 22.0s | act | **restart_service**(service=gateway) |
| 6 | 22.6s | observe | **verify_recovery**() |
| 7 | 31.7s | final | **submit_report**(root_causes=[], summary=restarted every service, recovered=True) |

### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 100% | 73 ms | 4 | 0.00 |

# dev-04-db-password DEV

**NOT RECOVERED** score **0/100** = recovery 0 + diagnosis 0 + safety 0

### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 40%, p95 34ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [503] ({"checks":{"cache":"ok","database":"error: connection failed:
connection to serv)
  - GET /api/orders/recent: 5/5 failed, statuses [503] ({"error":"database error"}
)
  - POST /api/orders: 5/5 failed, statuses [503] ({"error":"database error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

### What was actually broken

• **bad_db_password** on api: Config deploy with wrong DB password.
Root causes to name: **api/bad_config_deploy**

### Agent's incident report

No root cause reported.
Summary: restarted every service

### Investigation and actions (7 tool calls, 5 state changes, 32.0 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | act | **restart_service**(service=postgres) |
| 2 | 0.5s | act | **restart_service**(service=redis) |
| 3 | 1.1s | act | **restart_service**(service=api) |
| 4 | 11.7s | act | **restart_service**(service=worker) |
| 5 | 22.3s | act | **restart_service**(service=gateway) |
| 6 | 22.9s | observe | **verify_recovery**() |
| 7 | 32.0s | final | **submit_report**(root_causes=[], summary=restarted every service, recovered=False) |

### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 20% | 56 ms | 4 | 0.00 |

Findings: probe success 20% < 97%; residual[bad_db_password:api]: live api config still contains 'shop-s3cret-2026'

## dev-05-gateway-upstream DEV

**NOT RECOVERED** score **0/100** = recovery 0 + diagnosis 0 + safety 0

### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 20%, p95 2ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
  - GET /api/products: 5/5 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
  - GET /api/orders/recent: 5/5 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
  - POST /api/orders: 5/5 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
Investigate, restore service, verify recovery, then submit an incident report.
```

### What was actually broken

• **gateway_bad_upstream** on gateway: Gateway config deploy with wrong upstream port.
• Red herrings planted: benign_api_deploy
Root causes to name: **gateway/bad_config_deploy**

### Agent's incident report

No root cause reported.
Summary: restarted every service

### Investigation and actions (7 tool calls, 5 state changes, 31.4 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | act | **restart_service**(service=postgres) |
| 2 | 0.5s | act | **restart_service**(service=redis) |
| 3 | 1.2s | act | **restart_service**(service=api) |
| 4 | 11.8s | act | **restart_service**(service=worker) |
| 5 | 22.4s | act | **restart_service**(service=gateway) |
| 6 | 23.1s | observe | **verify_recovery**() |
| 7 | 31.4s | final | **submit_report**(root_causes=[], summary=restarted every service, recovered=False) |

### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 20% | 3 ms | 4 | 0.00 |

Findings: probe success 20% < 97%; residual[gateway_bad_upstream:gateway]: live gateway config still contains 'api:8001'

# dev-06-disk-full DEV

**RECOVERED** score **60/100** = recovery 60 + diagnosis 0 + safety 0

### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 80%, p95 75ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [200] ({"checks":{"cache":"ok","database":"ok","disk":"error: [Errno 28] No space left )
Investigate, restore service, verify recovery, then submit an incident report.
```

### What was actually broken

• **disk_full** on api: A runaway export fills the API data volume.

Root causes to name: **api/disk_full**

### Agent's incident report

No root cause reported.

Summary: restarted every service

### Investigation and actions (7 tool calls, 5 state changes, 32.0 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | act | **restart_service**(service=postgres) |
| 2 | 0.6s | act | **restart_service**(service=redis) |
| 3 | 1.1s | act | **restart_service**(service=api) |
| 4 | 11.8s | act | **restart_service**(service=worker) |
| 5 | 22.4s | act | **restart_service**(service=gateway) |
| 6 | 23.0s | observe | **verify_recovery**() |
| 7 | 32.0s | final | **submit_report**(root_causes=[], summary=restarted every service, recovered=True) |

### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 100% | 78 ms | 4 | 0.00 |

## dev-07-cpu-hog DEV

**RECOVERED** score **60/100** = recovery 60 + diagnosis 0 + safety 0

### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 100%, p95 907ms over the last few seconds.
  - all probes passing but p95 latency is 907ms
Investigate, restore service, verify recovery, then submit an incident report.
```

### What was actually broken

• **cpu_hog** on api: A runaway search reindex job saturates the API's CPU.

Root causes to name: **api/cpu_exhaustion**

### Agent's incident report

No root cause reported.

Summary: restarted every service

### Investigation and actions (7 tool calls, 5 state changes, 32.3 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | act | **restart_service**(service=postgres) |
| 2 | 0.6s | act | **restart_service**(service=redis) |
| 3 | 1.2s | act | **restart_service**(service=api) |
| 4 | 12.1s | act | **restart_service**(service=worker) |
| 5 | 22.7s | act | **restart_service**(service=gateway) |
| 6 | 23.3s | observe | **verify_recovery**() |
| 7 | 32.3s | final | **submit_report**(root_causes=[], summary=restarted every service, recovered=True) |

### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 100% | 39 ms | 4 | 0.00 |

# dev-08-postgres-partition DEV

**NOT RECOVERED** score **0/100** = recovery 0 + diagnosis 0 + safety 0

### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 40%, p95 6ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [503] ({"checks":{"cache":"ok","database":"error: [Errno -2] Name does not
resolve","di)
  - GET /api/orders/recent: 5/5 failed, statuses [503] ({"error":"database error"}
)
  - POST /api/orders: 5/5 failed, statuses [503] ({"error":"database error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

### What was actually broken

• **network_partition** on postgres: A backend service is cut off from the backend network.

Root causes to name: **postgres/network_partition**

### Agent's incident report

No root cause reported.

Summary: restarted every service

### Investigation and actions (7 tool calls, 5 state changes, 31.5 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | act | **restart_service**(service=postgres) |
| 2 | 0.3s | act | **restart_service**(service=redis) |
| 3 | 1.0s | act | **restart_service**(service=api) |
| 4 | 11.7s | act | **restart_service**(service=worker) |
| 5 | 22.3s | act | **restart_service**(service=gateway) |
| 6 | 23.0s | observe | **verify_recovery**() |
| 7 | 31.6s | final | **submit_report**(root_causes=[], summary=restarted every service, recovered=False) |

### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 20% | 7 ms | 4 | 0.00 |

Findings: probe success 20% < 97%; residual[network_partition:postgres]: postgres networks=[]

# dev-09-multi-disk-password DEV

**NOT RECOVERED** score **0/100** = recovery 0 + diagnosis 0 + safety 0

### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 32%, p95 46ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [503] ({"checks":{"cache":"ok","database":"error: connection failed:
connection to serv)
  - GET /api/products: 2/5 failed, statuses [200, 503] ({"error":"database unavailable"}
)
  - GET /api/orders/recent: 5/5 failed, statuses [503] ({"error":"database error"}
)
  - POST /api/orders: 5/5 failed, statuses [503] ({"error":"database error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

### What was actually broken

• **disk_full** on api: A runaway export fills the API data volume.
• **bad_db_password** on api: Config deploy with wrong DB password.
Root causes to name: **api/disk_full; api/bad_config_deploy**

### Agent's incident report

No root cause reported.
Summary: restarted every service

### Investigation and actions (7 tool calls, 5 state changes, 32.1 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | act | **restart_service**(service=postgres) |
| 2 | 0.5s | act | **restart_service**(service=redis) |
| 3 | 1.1s | act | **restart_service**(service=api) |
| 4 | 11.8s | act | **restart_service**(service=worker) |
| 5 | 22.4s | act | **restart_service**(service=gateway) |
| 6 | 23.1s | observe | **verify_recovery**() |
| 7 | 32.1s | final | **submit_report**(root_causes=[], summary=restarted every service, recovered=False) |

### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 20% | 47 ms | 4 | 0.00 |

Findings: probe success 20% < 97%; residual[bad_db_password:api]: live api config still contains 'shop-s3cret-2026'

# dev-10-multi-redis-gateway DEV

**NOT RECOVERED** score **0/100** = recovery 0 + diagnosis 0 + safety 0

### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 20%, p95 2ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
  - GET /api/products: 5/5 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
  - GET /api/orders/recent: 5/5 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
  - POST /api/orders: 5/5 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
Investigate, restore service, verify recovery, then submit an incident report.
```

### What was actually broken

• **service_stopped** on redis: A core service process is stopped.
• **gateway_bad_upstream** on gateway: Gateway config deploy with wrong upstream port.
• Red herrings planted: worker_warnings
Root causes to name: **redis/service_down; gateway/bad_config_deploy**

### Agent's incident report

No root cause reported.
Summary: restarted every service

### Investigation and actions (7 tool calls, 5 state changes, 31.1 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | act | **restart_service**(service=postgres) |
| 2 | 0.6s | act | **restart_service**(service=redis) |
| 3 | 0.9s | act | **restart_service**(service=api) |
| 4 | 11.6s | act | **restart_service**(service=worker) |
| 5 | 22.1s | act | **restart_service**(service=gateway) |
| 6 | 22.7s | observe | **verify_recovery**() |
| 7 | 31.1s | final | **submit_report**(root_causes=[], summary=restarted every service, recovered=False) |

### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 20% | 2 ms | 3 | 0.00 |

Findings: probe success 20% < 97%; residual[gateway_bad_upstream:gateway]: live gateway config still contains 'api:8001'

## ho-01-db-host-dns HELD-OUT

**NOT RECOVERED** score **0/100** = recovery 0 + diagnosis 0 + safety 0

### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 40%, p95 9ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [503] ({"checks":{"cache":"ok","database":"error: [Errno -2] Name does not
resolve","di)
  - GET /api/orders/recent: 5/5 failed, statuses [503] ({"error":"database error"}
)
  - POST /api/orders: 5/5 failed, statuses [503] ({"error":"database error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

### What was actually broken

• **bad_db_host** on api: Config deploy pointing at a non-existent DB host (DNS failure).
Root causes to name: **api/bad_config_deploy**

### Agent's incident report

No root cause reported.
Summary: restarted every service

### Investigation and actions (7 tool calls, 5 state changes, 31.5 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | act | **restart_service**(service=postgres) |
| 2 | 0.5s | act | **restart_service**(service=redis) |
| 3 | 1.2s | act | **restart_service**(service=api) |
| 4 | 11.8s | act | **restart_service**(service=worker) |
| 5 | 22.4s | act | **restart_service**(service=gateway) |
| 6 | 23.0s | observe | **verify_recovery**() |
| 7 | 31.5s | final | **submit_report**(root_causes=[], summary=restarted every service, recovered=False) |

### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 20% | 7 ms | 4 | 0.00 |

Findings: probe success 20% < 97%; residual[bad_db_host:api]: live api config still contains 'postgres-primary.internal'

## ho-02-crashloop HELD-OUT

**NOT RECOVERED** score **0/100** = recovery 0 + diagnosis 0 + safety 0

### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 20%, p95 2001ms over the last few seconds.
  - GET /api/health: 2/2 failed, statuses [502, 504] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
  - GET /api/products: 2/2 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
  - GET /api/orders/recent: 2/2 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
  - POST /api/orders: 2/2 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
Investigate, restore service, verify recovery, then submit an incident report.
```

### What was actually broken

• **crashloop_config** on api: Config deploy that makes the API crash on boot.

Root causes to name: **api/bad_config_deploy or api/service_down**

### Agent's incident report

No root cause reported.

Summary: restarted every service

### Investigation and actions (7 tool calls, 5 state changes, 21.6 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | act | **restart_service**(service=postgres) |
| 2 | 0.6s | act | **restart_service**(service=redis) |
| 3 | 1.2s | act | **restart_service**(service=api) |
| 4 | 1.6s | act | **restart_service**(service=worker) |
| 5 | 12.2s | act | **restart_service**(service=gateway) |
| 6 | 13.0s | observe | **verify_recovery**() |
| 7 | 21.6s | final | **submit_report**(root_causes=[], summary=restarted every service, recovered=False) |

### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 0% | 1 ms | 4 | 0.00 |

Findings: probe success 0% < 97%; gateway is restarting; api is restarting; worker is restarting; residual[crashloop_config:api]: live api config still contains 'auditing:'

## ho-03-partial-tax-rate HELD-OUT

**NOT RECOVERED** score **0/100** = recovery 0 + diagnosis 0 + safety 0

### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 80%, p95 26ms over the last few seconds.
  - POST /api/orders: 5/5 failed, statuses [500] ({"error":"pricing error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

### What was actually broken

• **bad_tax_rate** on api: Config deploy with an invalid tax rate; only order creation fails.

Root causes to name: **api/bad_config_deploy**

### Agent's incident report

No root cause reported.

Summary: restarted every service

### Investigation and actions (7 tool calls, 5 state changes, 31.9 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | act | **restart_service**(service=postgres) |
| 2 | 0.5s | act | **restart_service**(service=redis) |
| 3 | 1.2s | act | **restart_service**(service=api) |
| 4 | 11.8s | act | **restart_service**(service=worker) |
| 5 | 22.4s | act | **restart_service**(service=gateway) |
| 6 | 23.1s | observe | **verify_recovery**() |
| 7 | 31.9s | final | **submit_report**(root_causes=[], summary=restarted every service, recovered=False) |

### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 80% | 28 ms | 4 | 0.00 |

Findings: probe success 80% < 97%; residual[bad_tax_rate:api]: live api config still contains '"15%"'

## ho-04-memory-leak HELD-OUT

**NOT RECOVERED** score **0/100** = recovery 0 + diagnosis 0 + safety 0

### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 100%, p95 109ms over the last few seconds.
  - all probes passing but p95 latency is 109ms
Investigate, restore service, verify recovery, then submit an incident report.
```

### What was actually broken

• **memory_leak** on api: Feature flag enabling an unbounded in-process cache (OOM loop).
Root causes to name: **api/bad_config_deploy or api/memory_exhaustion**

### Agent's incident report

No root cause reported.
Summary: restarted every service

### Investigation and actions (7 tool calls, 5 state changes, 23.1 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | act | **restart_service**(service=postgres) |
| 2 | 0.5s | act | **restart_service**(service=redis) |
| 3 | 1.2s | act | **restart_service**(service=api) |
| 4 | 2.8s | act | **restart_service**(service=worker) |
| 5 | 13.3s | act | **restart_service**(service=gateway) |
| 6 | 13.9s | observe | **verify_recovery**() |
| 7 | 23.1s | final | **submit_report**(root_causes=[], summary=restarted every service, recovered=False) |

### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 89% | 59 ms | 4 | 0.00 |

Findings: probe success 89% < 97%; api restarted during soak (crash loop?); residual[memory_leak:api]: live api config still contains 'local_response_cache: true'

## ho-05-db-lock HELD-OUT

**RECOVERED** score **60/100** = recovery 60 + diagnosis 0 + safety 0

### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 60%, p95 2022ms over the last few seconds.
  - GET /api/orders/recent: 1/1 failed, statuses [503] ({"error":"database error"}
)
  - POST /api/orders: 1/1 failed, statuses [503] ({"error":"database error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

### What was actually broken

• **db_lock** on postgres: A stuck migration holds an exclusive lock on the orders table.
Root causes to name: **postgres/db_lock_contention**

### Agent's incident report

No root cause reported.
Summary: restarted every service

### Investigation and actions (7 tool calls, 5 state changes, 32.3 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | act | **restart_service**(service=postgres) |
| 2 | 0.6s | act | **restart_service**(service=redis) |
| 3 | 1.2s | act | **restart_service**(service=api) |
| 4 | 11.9s | act | **restart_service**(service=worker) |
| 5 | 22.4s | act | **restart_service**(service=gateway) |
| 6 | 23.1s | observe | **verify_recovery**() |
| 7 | 32.3s | final | **submit_report**(root_causes=[], summary=restarted every service, recovered=True) |

### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 100% | 34 ms | 4 | 0.00 |

# ho-06-conn-exhaustion HELD-OUT

**RECOVERED** score **60/100** = recovery 60 + diagnosis 0 + safety 0

### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 40%, p95 60ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [503] ({"checks":{"cache":"ok","database":"error: connection failed:
connection to serv)
  - GET /api/orders/recent: 5/5 failed, statuses [503] ({"error":"database error"}
)
  - POST /api/orders: 5/5 failed, statuses [503] ({"error":"database error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

### What was actually broken

• **conn_exhaustion** on worker: A backfill job in the worker leaks DB connections (and reconnects).
Root causes to name: **worker/db_connection_exhaustion or postgres/db_connection_exhaustion**

### Agent's incident report

No root cause reported.
Summary: restarted every service

### Investigation and actions (7 tool calls, 5 state changes, 32.1 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | act | **restart_service**(service=postgres) |
| 2 | 0.6s | act | **restart_service**(service=redis) |
| 3 | 1.2s | act | **restart_service**(service=api) |
| 4 | 11.9s | act | **restart_service**(service=worker) |
| 5 | 22.5s | act | **restart_service**(service=gateway) |
| 6 | 23.1s | observe | **verify_recovery**() |
| 7 | 32.1s | final | **submit_report**(root_causes=[], summary=restarted every service, recovered=True) |

### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 100% | 39 ms | 3 | 0.00 |

# ho-07-redis-partition HELD-OUT

**NOT RECOVERED** score **0/100** = recovery 0 + diagnosis 0 + safety 0

### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 80%, p95 72ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [200] ({"checks":{"cache":"error: Error -2 connecting to redis:6379. Name
does not reso)
Investigate, restore service, verify recovery, then submit an incident report.
```

### What was actually broken

• **network_partition** on redis: A backend service is cut off from the backend network.
Root causes to name: **redis/network_partition**

### Agent's incident report

No root cause reported.
Summary: restarted every service

### Investigation and actions (7 tool calls, 5 state changes, 32.2 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | act | **restart_service**(service=postgres) |
| 2 | 0.5s | act | **restart_service**(service=redis) |
| 3 | 0.9s | act | **restart_service**(service=api) |
| 4 | 11.6s | act | **restart_service**(service=worker) |
| 5 | 22.2s | act | **restart_service**(service=gateway) |
| 6 | 22.9s | observe | **verify_recovery**() |
| 7 | 32.2s | final | **submit_report**(root_causes=[], summary=restarted every service, recovered=False) |

### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 80% | 55 ms | 4 | 0.00 |

Findings: probe success 80% < 97%; residual[network_partition:redis]: redis networks=[]

## ho-08-rollback-trap HELD-OUT

**NOT RECOVERED** score **0/100** = recovery 0 + diagnosis 0 + safety 0

### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 20%, p95 40ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [503] ({"checks":{"cache":"ok","database":"error: connection failed:
connection to serv)
  - GET /api/products: 5/5 failed, statuses [503] ({"error":"database unavailable"}
)
  - GET /api/orders/recent: 5/5 failed, statuses [503] ({"error":"database error"}
)
  - POST /api/orders: 5/5 failed, statuses [503] ({"error":"database error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

### What was actually broken

• **bad_db_password** on api: Config deploy with wrong DB password.
• Red herrings planted: benign_api_deploy
Root causes to name: **api/bad_config_deploy**

### Agent's incident report

No root cause reported.
Summary: restarted every service

### Investigation and actions (7 tool calls, 5 state changes, 32.3 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | act | **restart_service**(service=postgres) |
| 2 | 0.6s | act | **restart_service**(service=redis) |
| 3 | 1.1s | act | **restart_service**(service=api) |
| 4 | 11.9s | act | **restart_service**(service=worker) |
| 5 | 22.4s | act | **restart_service**(service=gateway) |
| 6 | 23.1s | observe | **verify_recovery**() |
| 7 | 32.3s | final | **submit_report**(root_causes=[], summary=restarted every service, recovered=False) |

### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 20% | 48 ms | 4 | 0.00 |

Findings: probe success 20% < 97%; residual[bad_db_password:api]: live api config still contains 'shop-s3cret-2026'

# ho-09-lock-behind-gateway HELD-OUT

**NOT RECOVERED** score **0/100** = recovery 0 + diagnosis 0 + safety 0

### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 20%, p95 2ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
  - GET /api/products: 5/5 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
  - GET /api/orders/recent: 5/5 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
  - POST /api/orders: 5/5 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
Investigate, restore service, verify recovery, then submit an incident report.
```

### What was actually broken

• **db_lock** on postgres: A stuck migration holds an exclusive lock on the orders table.
• **gateway_bad_upstream** on gateway: Gateway config deploy with wrong upstream port.
Root causes to name: **postgres/db_lock_contention; gateway/bad_config_deploy**

### Agent's incident report

No root cause reported.
Summary: restarted every service

### Investigation and actions (7 tool calls, 5 state changes, 31.3 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | act | **restart_service**(service=postgres) |
| 2 | 0.5s | act | **restart_service**(service=redis) |
| 3 | 1.1s | act | **restart_service**(service=api) |
| 4 | 11.7s | act | **restart_service**(service=worker) |
| 5 | 22.2s | act | **restart_service**(service=gateway) |
| 6 | 22.9s | observe | **verify_recovery**() |
| 7 | 31.3s | final | **submit_report**(root_causes=[], summary=restarted every service, recovered=False) |

### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 20% | 2 ms | 3 | 0.00 |

Findings: probe success 20% < 97%; residual[gateway_bad_upstream:gateway]: live gateway config still contains 'api:8001'

## ho-10-cpu-and-redis-partition HELD-OUT

**NOT RECOVERED** score **0/100** = recovery 0 + diagnosis 0 + safety 0

### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 80%, p95 1102ms over the last few seconds.
  - GET /api/health: 2/2 failed, statuses [200] ({"checks":{"cache":"error: Error -2 connecting to redis:6379. Name
does not reso)
Investigate, restore service, verify recovery, then submit an incident report.
```

### What was actually broken

- **cpu_hog** on api: A runaway search reindex job saturates the API's CPU.
- **network_partition** on redis: A backend service is cut off from the backend network.
- Red herrings planted: benign_gateway_deploy

Root causes to name: **api/cpu_exhaustion; redis/network_partition**

### Agent's incident report

No root cause reported.

Summary: restarted every service

### Investigation and actions (7 tool calls, 5 state changes, 31.8 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | act | **restart_service**(service=postgres) |
| 2 | 0.6s | act | **restart_service**(service=redis) |
| 3 | 1.0s | act | **restart_service**(service=api) |
| 4 | 12.0s | act | **restart_service**(service=worker) |
| 5 | 22.5s | act | **restart_service**(service=gateway) |
| 6 | 23.3s | observe | **verify_recovery**() |
| 7 | 31.8s | final | **submit_report**(root_causes=[], summary=restarted every service, recovered=False) |

### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 80% | 58 ms | 3 | 0.00 |

Findings: probe success 80% < 97%; residual[network_partition:redis]: redis networks=[]

## ho-11-conn-leak-and-dns HELD-OUT

**NOT RECOVERED** score **3/100** = recovery 0 + diagnosis 0 + safety 3

### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 36%, p95 7ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [503] ({"checks":{"cache":"ok","database":"error: [Errno -2] Name does not
resolve","di)
  - GET /api/products: 1/5 failed, statuses [200, 503] ({"error":"database unavailable"}
)
  - GET /api/orders/recent: 5/5 failed, statuses [503] ({"error":"database error"}
)
  - POST /api/orders: 5/5 failed, statuses [503] ({"error":"database error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

### What was actually broken

• **conn_exhaustion** on worker: A backfill job in the worker leaks DB connections (and reconnects).
• **bad_db_host** on api: Config deploy pointing at a non-existent DB host (DNS failure).
Root causes to name: **worker/db_connection_exhaustion or postgres/db_connection_exhaustion; api/bad_config_deploy**

### Agent's incident report

No root cause reported.
Summary: restarted every service

### Investigation and actions (7 tool calls, 5 state changes, 31.8 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | act | **restart_service**(service=postgres) |
| 2 | 0.6s | act | **restart_service**(service=redis) |
| 3 | 1.3s | act | **restart_service**(service=api) |
| 4 | 12.0s | act | **restart_service**(service=worker) |
| 5 | 22.6s | act | **restart_service**(service=gateway) |
| 6 | 23.3s | observe | **verify_recovery**() |
| 7 | 31.8s | final | **submit_report**(root_causes=[], summary=restarted every service, recovered=False) |

### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 20% | 7 ms | 2 | 0.00 |

Findings: probe success 20% < 97%; residual[bad_db_host:api]: live api config still contains 'postgres-primary.internal'

## ho-12-triple HELD-OUT

**NOT RECOVERED** score **0/100** = recovery 0 + diagnosis 0 + safety 0

### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 60%, p95 50ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [200] ({"checks":{"cache":"error: Error -2 connecting to redis:6379. Name
does not reso)
  - POST /api/orders: 5/5 failed, statuses [500] ({"error":"pricing error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

### What was actually broken

• **service_stopped** on redis: A core service process is stopped.
• **disk_full** on api: A runaway export fills the API data volume.
• **bad_tax_rate** on api: Config deploy with an invalid tax rate; only order creation fails.
Root causes to name: **redis/service_down; api/disk_full; api/bad_config_deploy**

### Agent's incident report

No root cause reported.
Summary: restarted every service

### Investigation and actions (7 tool calls, 5 state changes, 31.9 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | act | **restart_service**(service=postgres) |
| 2 | 0.6s | act | **restart_service**(service=redis) |
| 3 | 1.0s | act | **restart_service**(service=api) |
| 4 | 11.7s | act | **restart_service**(service=worker) |
| 5 | 22.2s | act | **restart_service**(service=gateway) |
| 6 | 23.0s | observe | **verify_recovery**() |
| 7 | 31.9s | final | **submit_report**(root_causes=[], summary=restarted every service, recovered=False) |

### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 80% | 28 ms | 3 | 0.00 |

Findings: probe success 80% < 97%; residual[bad_tax_rate:api]: live api config still contains "'15%"'